

# PySRCar: Learning Planar Vehicle Dynamics in MuJoCo with Symbolic Regression

Aditya Pillai

December 15, 2025

## 1 Overview

This short report summarizes a MuJoCo simulation for a planar “car” (a rigid block that translates in  $x, y$  and rotates in yaw) and an experiment using PySR (symbolic regression) to learn continuous-time dynamics from logged rollouts. The goal was to (i) generate clean state-control trajectories under a simple tracking controller, and (ii) fit explicit closed-form expressions for  $\dot{x}$ ,  $\dot{y}$ ,  $\dot{\psi}$  and the accelerations using only the logged data.

## 2 Simulation setup (MuJoCo)

The system is defined in `models/planar_block.xml`. The body has three joints: two slide joints for planar translation and one hinge joint about the  $z$  axis for yaw. The state used throughout the code is

$$\mathbf{s} = [x, y, \psi, v_x, v_y, \omega]^\top,$$

where  $\psi$  is yaw and  $\omega$  is yaw rate (logged as `wyaw`). Control is parameterized as a body-forward thrust plus a yaw torque:

$$\mathbf{u} = [T, \tau]^\top,$$

with saturation limits given by the actuator `ctrlrange` in the XML (and mirrored in code).

In the simulator (`sim/run_planar_block.py`), the forward thrust  $T$  is projected into world-frame forces along the slide joints using the current yaw:

$$F_x = T \cos \psi, \quad F_y = T \sin \psi,$$

and the actuator inputs are set as `ctrl` =  $[F_x, F_y, \tau]$ .

## 3 Controller and data logging

A simple reference generator produces a circular trajectory of radius  $R$  with constant tangential speed  $v_{\text{ref}}$ :

$$x_{\text{ref}}(t) = R \cos(\omega t), \quad y_{\text{ref}}(t) = R \sin(\omega t), \quad \psi_{\text{ref}}(t) = \omega t + \frac{\pi}{2}, \quad \omega = \frac{v_{\text{ref}}}{R}.$$

The controller uses a longitudinal position error (projected onto the body-forward axis) plus a speed term for thrust, and a PD term for yaw torque:

$$T = k_v(v_{\text{ref}} - v) + k_p e_{\text{long}}, \quad \tau = k_\psi(\psi_{\text{ref}} - \psi) - k_d \omega,$$

followed by clipping to the configured limits.

Each rollout is logged to `data/run_*.csv` with columns: `t`, `x`, `y`, `yaw`, `vx`, `vy`, `wyaw`, `v`, `thrust`, `yaw_torque`. Figure 1 shows an example logged rollout (used for the results below).



Figure 1: Example rollout: planar trajectory, forward speed, yaw, and control inputs.

## 4 Dynamics learning with PySR

The learning code is in `sim/pysr_dynamics.py`. The feature vector for regression is

$$\mathbf{x} = [x, y, \psi, v_x, v_y, \omega, T, \tau]^\top,$$

and the target is a chosen component of  $\dot{\mathbf{s}}$ . Derivatives are formed as:

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{\psi} = \omega,$$

and accelerations are computed using finite differences, e.g.

$$\dot{v}_x(t_i) \approx \frac{v_x(t_{i+1}) - v_x(t_i)}{\Delta t}.$$

`PySRRegressor` is then fit for each target derivative using a small operator library (binary: `+`, `-`, `*`, `/`, `^`; unary: `sin`, `cos`, `exp`, `log`).

## 5 Results

### 5.1 Recovered kinematics

For the first three derivatives, the regression recovers the expected identities:

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{\psi} = \omega.$$

This is a useful sanity check that the data pipeline and feature ordering are consistent.

## 5.2 Learned acceleration models

I was also able to identify the minimum-loss expressions from the most recent training artifacts in **outputs/** for  $\dot{v}_x$ ,  $\dot{v}_y$ , and  $\dot{\omega}$ . These expressions are not as obvious as the identities above and are a bit more complex. Thus, they are not included in the report.

## 6 Discussion and next steps

This pipeline demonstrates end-to-end data generation (MuJoCo  $\rightarrow$  CSV logs) and symbolic regression of dynamics (CSV  $\rightarrow$  explicit formulas). The learned expressions recover the kinematic identities and produce a reasonable model for our robotic system.

For improved acceleration models, the main next steps are: (i) collect more diverse rollouts (different radii/speeds, transients, random thrust/torque inputs) to excite the dynamics, (ii) add explicit regularization toward physically meaningful structure (e.g., dependence on  $\cos\psi, \sin\psi$ ), and (iii) validate learned models on held-out trajectories.